

Comparação Entre Algoritmos de Escalonamento

Breno C. R. Nakamura¹, Estevan T. Santos¹, Gabriel L. Bonavina¹,
Jonas L. Durão¹, Natália V. A. do Val¹, Vinicius A. L. Andrade¹

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
Av. Cesare Monsueto Giulio Lattes, 1201 – 12247-014 – São José dos Campos, SP

Abstract. *Building on the knowledge acquired during the Operating Systems (OS) course, this project aims to deepen practical understanding of the scheduling algorithms discussed in class, from installing MINIX 3.4.0rc6 to modifying the scheduling algorithms themselves. First, we will describe the system installation process and simple modifications that helped us better understand the OS structure. Next, we will modify the MINIX source code to implement the scheduling algorithms discussed in class and evaluate the performance of each one. Finally, we will discuss the results obtained and the project's conclusions.*

Resumo. *Permeados pelo conhecimento adquirido durante a disciplina de Sistemas Operacionais (SO), este projeto tem como objetivo aprofundar o conhecimento prático sobre algoritmos de escalonamento discutidos em sala, desde a instalação do MINIX 3.4.0rc6, até a modificação dos algoritmos de escalonamento propriamente. Primeiramente, detalharemos o processo de instalação do sistema e modificações simples que nos permitiram compreender mais sobre a estrutura do SO. Em seguida, realizaremos modificações no código-fonte do MINIX para implementar os algoritmos de escalonamento discutidos em sala, e avaliaremos o desempenho de cada um deles. Por fim, discutiremos os resultados obtidos e as conclusões do projeto.*

O código-fonte completo deste projeto pode ser encontrado no repositório do Github: <https://github.com/brenonak/minix>

1. Introdução

O MINIX 3 é um sistema operacional de código aberto criado por Andrew S. Tanenbaum. Ele foi projetado para ser confiável e flexível, sendo utilizado para fins didáticos [Tanenbaum and Woodhull 2008]. Sua arquitetura é baseada em microkernel, na qual apenas as funções mais básicas, como o gerenciamento de interrupções e comunicação entre processos, são executadas no núcleo do sistema. Os demais componentes, como drivers, gerenciamento de arquivos e o escalonador, operam separados do núcleo, como processos independentes no espaço do usuário. Essa separação ajuda a evitar que falhas em um serviço comprometam o sistema inteiro, além de facilitar o estudo e modificação de seu código-fonte.

Este projeto utiliza a versão MINIX 3.4.0rc6 com o objetivo de aplicar na prática os conceitos teóricos da disciplina de Sistemas Operacionais. O trabalho explora o funcionamento interno do sistema por meio de sua instalação em ambiente virtualizado, alteração de chamadas no kernel e, principalmente, através da implementação e análise comparativa de desempenho de diferentes algoritmos de escalonamento de processos sob variadas cargas de trabalho.

2. Instalação

A primeira etapa deste projeto consistiu na instalação e configuração inicial do MINIX na máquina virtual. Para isso, foi necessário seguir o manual de instruções apresentado no enunciado. Imagens da instalação podem ser encontradas nas Figuras 1 e 2.

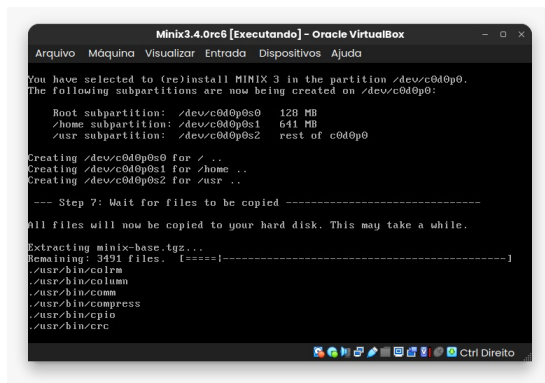


Figura 1. Etapa 7 de instalação do MINIX.

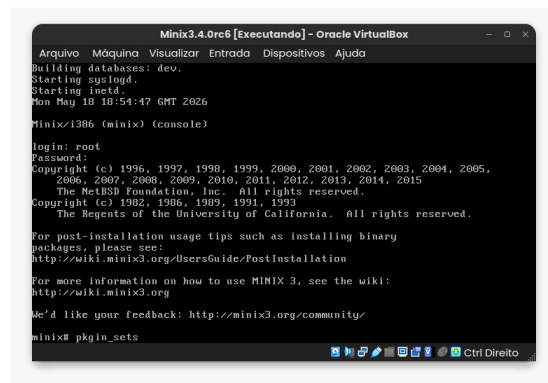


Figura 2. Atualização dos pacotes do MINIX.

Além da instalação do MINIX, também foi necessário fazer a configuração de rede no *Virtual Box*, ajustando a política de endereçamento MAC para incluir apenas os endereços MAC de placas em rede NAT.

Além disso, foi necessário fazer o *clone* do repositório no ambiente virtual, para que fosse possível fazer as modificações que serão detalhadas em seções futuras deste relatório. Para isso, foi necessário configurar um Token de Acesso Pessoal do Github, o que nos permitiu executar os comandos do *git* no ambiente virtual. Encontramos dificuldades nessa parte, já que no ambiente virtual não é possível utilizar os comandos de cópia e colagem. Para contornar esse problema, criamos um arquivo `clonar.sh` e um arquivo `meu_token.txt`, que contém o token de acesso pessoal do Github, e, ao rodar o arquivo `clonar.sh`, o repositório é clonado no ambiente virtual. Optamos por esta abordagem pois é uma forma de garantir que o token do Github esteja correto no momento da execução do script.

Por fim, faz-se necessário ressaltar as configurações de hardware da máquina virtual. Para a execução deste projeto utilizamos uma máquina virtual com 1 *core* de processador, 1GB de memória RAM e 4GB de HD dinamicamente alocado.

3. Mudança nos Banners

Para a mudança no banner durante *boot* e *poweroff*, foi necessário navegar no diretório `minix/minix/kernel/main.c`. Na função `announce()`, foram acrescentados *prints* para reproduzir o banner indicado no enunciado. Além disso, na função `prepare_shutdown()`, também inserimos *prints* para a impressão do banner. Os resultados podem ser encontrados nas Figuras 3 e 4.

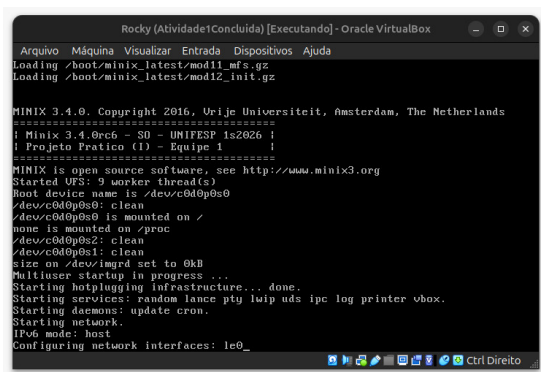


Figura 3. Resultado do banner de *boot*.

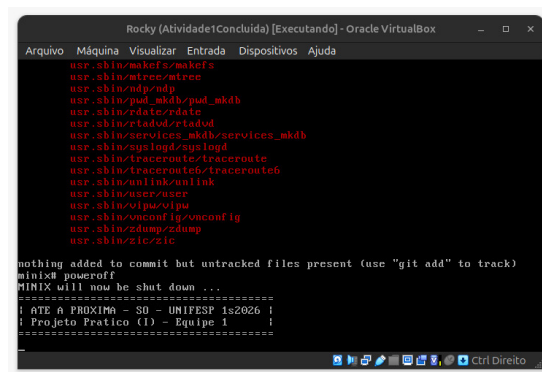


Figura 4. Resultado do banner de *poweroff*.

Por fim, para a modificação do banner de *login*, foi necessário navegar pelos diretórios `minix/etc/motd`, onde inserimos o banner indicado pelo enunciado. O resultado da modificação pode ser encontrado na Figura 5.

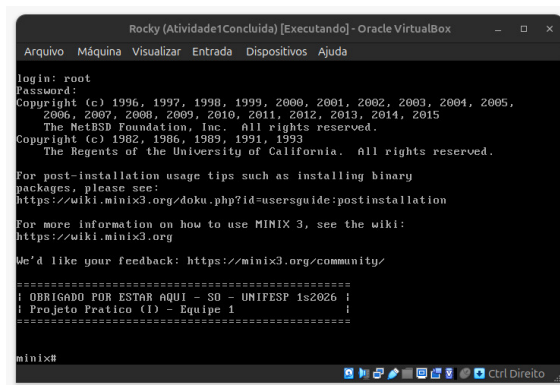


Figura 5. Resultado do banner de *login*.

4. Mudança `exec.c`

Para registrar a execução de processos exibindo o comando e seu caminho absoluto, a implementação foi dividida entre o *Virtual File System* (VFS) e o *Process Manager* (PM), respeitando a modularidade da arquitetura do MINIX [MINIX 3 Project].

Inicialmente, adicionou-se o campo `execpath` à estrutura compartilhada `exec_info` (no arquivo `libexec.h`) como um campo de comunicação entre o VFS e o PM. No VFS (`vfs/exec.c`), logo após o carregamento do executável, o caminho absoluto armazenado na variável `finalexec` é copiado para `execi.args.execpath` utilizando a função `strncpy`. Em seguida, o VFS aciona o PM por meio de `libexec.pm.newexec`.

A exibição da mensagem foi centralizada no PM (`pm/exec.c`). Na rotina do `newexec`, os dados preenchidos pelo VFS são recebidos via `sys_datacopy`. Após a cópia dos argumentos para a memória do PM, inseriu-se um `printf` para exibir a mensagem Executando: `<caminho/comando>`. O resultado dessa modificação pode ser encontrado na Figura 6.

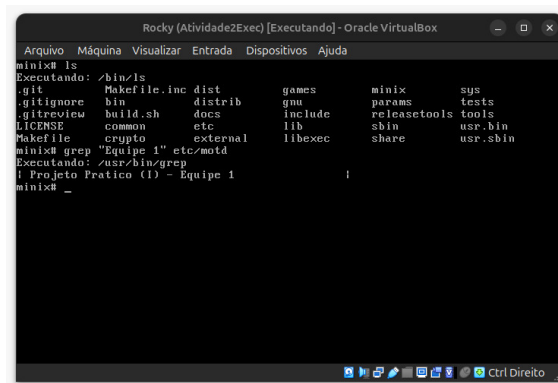


Figura 6. Resultado da modificação do `exec.c`.

Por fim, é válido destacar que todas os caminhos de arquivos modificados durante esta fase do projeto foram encontrados por meio da documentação oficial do MINIX [MINIX 3 Project].

5. Descrição do Exercício Proposto

Este projeto avalia o desempenho de diferentes algoritmos de escalonamento no MINIX 3.4.0rc6. Além do algoritmo padrão, o código-fonte do sistema foi modificado para implementar três escalonadores diferentes: *Round-Robin*, Múltiplas Filas com *Quantum* Exponencial e *First-Come, First-Served* (FCFS). A eficiência de cada escalonador foi testada sob perfis *CPU-bound*, com cargas de 1 milhão de operações por processo, e perfis *I/O-bound*, com 10 mil operações. A avaliação desenvolvida utilizou o programa `teste_processos.c`, submetendo o sistema a escalas de 10, 50, 100 e 200 processos simultâneos.

6. Descrição dos Algoritmos de Escalonamento

Para fundamentar as políticas implementadas neste projeto, é essencial estabelecer a base teórica do escalonamento de processos. Em sistemas multiprogramados, o escalonador é o componente do sistema operacional responsável por decidir qual tarefa terá acesso à CPU. Os algoritmos que guiam essa escolha dividem-se em duas categorias: os não preemptivos, nos quais a tarefa executa ininterruptamente até ser bloqueada ou liberar o processador de forma voluntária; e os preemptivos, que forçam a interrupção do processo após um intervalo de tempo máximo predefinido, conhecido como *quantum* [Tanenbaum 2016, p. 103–106]. Os algoritmos abordados a seguir englobam ambas as categorias: o modelo padrão do MINIX, o *Round-Robin* e as Múltiplas Filas com *Quantum* Exponencial operam de forma preemptiva, enquanto o FCFS adota o modelo não preemptivo.

6.1. Algoritmo Padrão do MINIX

O algoritmo de escalonamento padrão do MINIX 3 utiliza um escalonador preemptivo baseado em múltiplas filas de prioridade com ajuste dinâmico. O sistema gerencia 16 filas (0 a 15), nas quais o nível 0 possui a maior prioridade e o nível 15, a menor. O escalonador sempre seleciona o processo localizado no topo da fila de maior prioridade que não esteja vazia. As filas superiores, de 0 a 6, são destinadas a processos críticos do núcleo e a servidores essenciais do sistema. Já os processos de usuário são alocados entre

as filas 7 (`USER_Q`) e 14 (`MIN_USER_Q`). A fila 15 (`IDLE_Q`), de menor prioridade, é reservada ao processo ocioso do sistema, executado apenas quando não há nenhuma outra instrução pendente no processador.

Devido à arquitetura de microkernel do MINIX 3, o escalonamento ocorre tanto no espaço de núcleo quanto no espaço de usuário. No espaço de núcleo, implementado principalmente em `/minix/kernel/proc.c`, encontram-se os mecanismos de baixo nível, fornecendo as operações `enqueue()` e `dequeue()`, responsáveis pela manipulação das estruturas de dados, além da função `pick_proc()`, responsável por selecionar o próximo processo a receber a CPU com base na prioridade estrita.

Por outro lado, a política de escalonamento é implementada no espaço de usuário no diretório `/minix/servers/sched/`, com destaque para o arquivo `schedule.c`, onde o servidor de escalonamento `sched` gerencia o tempo de execução dos processos. Internamente, cada fila utiliza o algoritmo Round-Robin, no qual cada processo recebe uma fatia limitada de tempo de CPU (*quantum*). O servidor `sched` reduz a prioridade de processos pesados (*CPU-bound*) que esgotam seu *quantum* e promove processos interativos (*I/O-bound*) por meio do mecanismo de *aging*, implementado na função `balance_queues()`, garantindo maior responsividade e evitando inanição.

6.2. Round-Robin (RR)

O algoritmo de escalonamento por chaveamento circular (ou *Round-Robin*), é um dos algoritmos mais simples e antigos devido à sua fácil implementação. Seu funcionamento gira em torno da definição de uma fatia de tempo de CPU (*quantum*), em que o processo que está na CPU terá um tempo limitado de uso e, ao final deste tempo, haverá preempção e um novo processo será escolhido para ser executado [Tanenbaum 2016, p. 109–110].

A implementação concentrou-se em `minix/servers/sched/schedule.c`, sem alterar o *kernel*. O *quantum* definido foi de $\frac{1000}{n} ms$, com n dado por `count_ready_user_queue()` (processos de usuário prontos em `USER_Q`, via `sys_getinfo(GET_PROC)` e `proc_is_runnable()`). Foram incluídos `<minix/timers.h>` e `"kernel/proc.h"` e criadas `apply_variable_quantum()` e `reschedule_all_user_quantum()`; em `do_start_scheduling()`, `do_stop_scheduling()` e `do_noquantum()` o tempo de CPU é recalculado e aplicado com `sys_schedule()`. Processos de sistema (`is_system_proc`) mantêm prioridade própria; os restantes ficam fixos em `USER_Q`.

Para a implementação do RR ser feita sem mudar as funções `enqueue()`, `dequeue()` e `pick_proc()` (`minix/kernel/proc.c`), todos os usuários são forçados a adotar a fila `USER_Q` 7 em `do_start_scheduling()` e `do_nice()`; a rotação na cauda ao expirar o *quantum* ou ao desbloquear por E/S equivale ao *Round Robin* entre processos nessa fila. Em `minix/servers/pm/utility.c`, `nice_to_priority()` passa sempre `*new_q = USER_Q`, pelo que `nice()` deixa de distribuir processos pelas 16 filas. **Confuso**

Removeu-se a política MLFQ no SCHED em `do_noquantum()` eliminou-se `rmp->priority += 1`. `balance_queues()` e `init_scheduling()` ficaram vazias e em `main.c` deixou de se chamar `balance_queues()` nas notificações do CLOCK. Ao expirar o *quantum*, o processo permanece em `USER_Q`, recalcula a fatia com

`apply_variable_quantum()` e reagenda-se com `schedule_process_local()` (reinserção na cauda no kernel via `sched_proc`).

6.3. Múltiplas Filas com Quantum Exponencial

O segundo escalonamento implementado altera o algoritmo de múltiplas filas do MINIX 3, que originalmente adota um *quantum* fixo por processo de usuário. A modificação baseia-se no sistema CTSS, que operava no IBM 7094 ([Tanenbaum 2016, p. 111]), partindo do pressuposto de que conceder fatias de CPU progressivamente maiores reduz o *overhead* causado pelo excesso de trocas de contexto. Para isso, o projeto define que, ao esgotar seu tempo de CPU alocado, o processo é rebaixado à fila inferior, recebendo um novo *quantum* que cresce em potências de dois. Na última fila, os processos remanescentes são escalonados segundo a política *Round-Robin* com o *quantum* correspondente àquele nível.

Para implementar as alterações propostas, as modificações foram feitas no arquivo `minix/servers/sched/schedule.c`. Vale ressaltar que as novas regras de escalonamento aplicam-se exclusivamente aos processos de usuário, ou seja, aqueles que estão em filas acima de `USER_Q` 7. Para uma implementação coerente, definiu-se uma nova fatia de tempo inicial correspondente a 10% do valor padrão que o MINIX 3.4.0rc6 define para processos de usuário. Isso foi feito a partir da modificação do valor de `DEFAULT_USER_TIME_SLICE` para o valor de 20 no lugar de 200. Isso permitiu verificar nos resultados a influência do crescimento progressivo do tempo de CPU para processos longos.

Com o objetivo de desenvolver essa lógica, a função `user_quantum_for_priority` foi criada para calcular, em potências de dois, o *quantum* correspondente a cada nível de fila. O controle do tempo de CPU atribuído a cada processo, por sua vez, é atualizado pela função `update_user_quantum`. A chamada dessa função ocorre em situações distintas: em `do_start_scheduling()`, para inicializar o processo com o *quantum* base; em `do_noquantum()`, para atualizar o *quantum* no momento da preempção por esgotamento do tempo de CPU do processo; no caso `SCHEDULING_INHERIT`, para assegurar que o filho, criado na mesma fila de prioridade que o pai, tenha seu *quantum* corretamente definido; e, por fim, em `do_nice()`, para que a mudança de prioridade ajuste a fatia de tempo adequada.

Para assegurar o comportamento definido no projeto, o rebalanceamento das filas executado periodicamente em determinados ciclos de *clock* foi desativado. Para isso, em `init_scheduling()` os trechos que fazem a chamada de `sys_setalarm(balance_timeout)` foram comentados, de modo que em `minix/servers/sched/main.c` não haja a invocação de `balance_queues()`.

6.4. First-Come, First-Served (FCFS)

O algoritmo *First-Come, First-Served* atende os processos estritamente na ordem de chegada, sem preempção por tempo. Sua implementação no MINIX 3 exigiu desativar a prioridade dinâmica de múltiplas filas e as interrupções temporais para as tarefas de usuário, garantindo que executem de forma exclusiva até sua finalização ou bloqueio por E/S.

No espaço de núcleo, as modificações foram realizadas no arquivo `/minix/kernel/proc.c`, onde a preempção foi desabilitada na função

`proc_no_time()`. A chamada `notify_scheduler(p)`, que efetuava o dequeue do processo ao fim do tempo, foi substituída pela renovação contínua da sua fatia de tempo (`p->p_cpu_time_left`). Essa abordagem mantém a CPU com o processo atual sem quebrar os cálculos estatísticos internos do microkernel com um *quantum* infinito.

No espaço de usuário, as alterações foram realizadas no arquivo `/minix/servers/sched/schedule.c`. A estabilidade do sistema foi assegurada mantendo as filas 0 a 6 inalteradas para os servidores críticos. O FCFS foi aplicado exclusivamente aos processos do usuário. Na função `do_start_scheduling()`, todo processo com prioridade igual ou superior a 7 passou a ser alocado forçadamente em uma única fila (`USER_Q`). Com todos os processos de usuário concentrados nessa fila e sem variação de prioridade, os mecanismos de rebaixamento implementados em `do_noquantum()` e de promoção por *aging* implementados em `balance_queues()` foram completamente removidos. Por fim, para evitar interferências externas, a função `do_nice()` foi modificada para retornar `OK` imediatamente, impedindo a alteração manual de prioridades pelos usuários.

7. Resultados Obtidos e Discussão

Para garantir a robustez estatística de nossos resultados, realizaram-se 5 execuções para cada algoritmo, e calcularam-se a média e o desvio-padrão de cada um deles. Ao final das execuções, foi desenvolvido um gráfico de barras para cada classe de processos, no qual o eixo X representa o número de processos e o eixo Y representa o tempo de execução médio. Esses resultados são apresentados na Figura 7 e na Figura 8. Neles são apresentados em azul o algoritmo padrão do MINIX, em amarelo o FCFS, em verde o *Round-Robin* (RR) e em vermelho o Múltiplas Filas com *Quantum* Exponencial (MF).

Também é válido ressaltar que todos os testes de desempenho foram executados no mesmo computador, o qual foi mantido sem a execução de tarefas paralelas ou uso pessoal, de modo que os resultados obtidos fossem consistentes. A configuração do computador utilizado foi um processador 12th Gen Intel(R) Core(TM) i5-12500H com 16GB de RAM, com o Windows 11 instalado. Por fim, a versão da Oracle VirtualBox é a 7.2.6.

7.1. Resultados para testes de processos *CPU-bound*

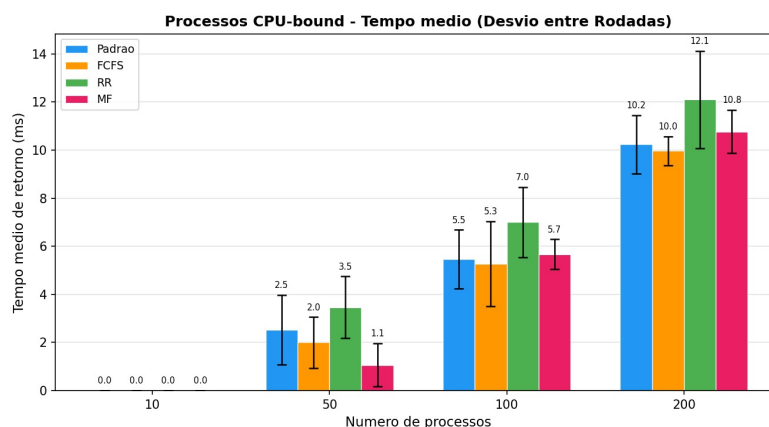


Figura 7. Resultados obtidos para processos *CPU-bound*.

Partindo pela análise dos resultados para os processos *CPU-bound*, apresentado na Figura 7, observa-se, de maneira geral, que o *Round-Robin* teve o pior desempenho dentre todos os métodos de escalonamento a partir dos testes com 50 processos. Partindo pela análise dos resultados para os processos *CPU-bound*, apresentados na Figura 7, observa-se, de maneira geral, que o algoritmo Round-Robin (RR) teve o pior desempenho dentre todos os métodos a partir dos testes com 50 processos. Isso ocorre porque o RR força a preempção baseada puramente no tempo (*quantum*). Como os processos *CPU-bound* realizam cálculos contínuos, eles sempre esgotam seu *quantum* antes de terminarem a tarefa, forçando o sistema operacional a realizar um número excessivo de trocas de contexto. Essa sobrecarga de salvar e restaurar o estado dos processos repetidas vezes eleva o tempo total de execução.

Avaliando cada teste individualmente, verifica-se que, com uma carga base de 10 processos, o tempo de retorno registrado é de 0.0 ms para todos os algoritmos. É importante destacar que esse valor não indica um tempo de execução nulo, mas sim que a duração foi tão curta que ficou abaixo do limite de resolução rastreável pelo mecanismo de medição do sistema. Todos os algoritmos foram capazes de concluir as tarefas de forma quase instantânea.

As divergências entre as políticas passaram a ser nítidas à medida que a carga aumentou. Para 50 processos, o algoritmo MF destaca-se com o menor tempo médio de retorno (1.1 ms). Nesse algoritmo, os processos *CPU-bound* que esgotam o *quantum* são rebaixados de fila, mas ganham fatias de tempo exponencialmente maiores. Assim, para 50 processos, a sobrecarga inicial de trocas de contexto nas filas superiores é compensada pela acomodação desses processos em filas mais baixas. Dessa forma, tarefas de uso intensivo de CPU puderam ser executadas por fatias de tempo maiores sem novas trocas de contexto, finalizando a execução de maneira muito mais rápida.

Por outro lado, em cenários de alta concorrência (100 e 200 processos), o MF perde eficiência, ficando à frente apenas do RR, em razão do grande volume de trocas de contexto exigido inicialmente. Esse *overhead* inicial maior não conseguiu ser compensado pela estabilidade posterior nas filas inferiores. Além disso, as últimas filas, que executam processos únicos por longos períodos, são suscetíveis a acumular uma extensa sequência de processos no estado de pronto, gerando longos tempos de espera e o risco de *starvation*.

Nesses cenários extremos, os algoritmos FCFS e o Padrão do MINIX demonstram a melhor escalabilidade. No teste mais crítico, com 200 processos, o FCFS obtém a melhor marca com 10.0 ms, empatado tecnicamente com o escalonador Padrão (10.2 ms), considerando a margem de erro. O desempenho superior do FCFS nestas condições justifica-se teoricamente pela sua natureza não preemptiva: ao ganhar a CPU, um processo limitado por processamento executa suas operações matemáticas até o fim, sem interrupções forçadas pelo timer do sistema. Isso elimina o *overhead* gerado por trocas de contexto, permitindo que a CPU dedique 100% dos seus ciclos à execução útil do teste.

Em contrapartida, o algoritmo Round Robin (RR) sofre o maior impacto sob carga pesada, atingindo a pior média (12.1 ms) e exibindo a maior variância (desvio-padrão) entre as rodadas. Oposto ao cenário do FCFS, essa degradação ocorre justamente pelo excesso de trocas de contexto. Devido ao número de processos na fila, pois o *quantum*

com fração justa será muito menor ($\frac{1000}{200} = 5\text{ ms}$), resultando em um processo que em execução ficará pouco tempo na CPU, sendo necessário muitas trocas de contexto.

Por fim, um ponto válido a ser levantado acerca da Figura 7 é o comportamento do algoritmo padrão do MINIX no cenário extremo de 200 processos. Embora apresente a segunda melhor média (10.2 ms), sua barra de desvio-padrão indica que, em algumas execuções, ele foi capaz de atingir performances na casa dos 9.0 ms, superando o FCFS. Isso demonstra que as otimizações internas do escalonador dinâmico do MINIX podem, em rodadas específicas, favorecer a execução em lote. Contudo, ao analisar a sobreposição das margens de erro, conclui-se que o FCFS e o algoritmo Padrão possuem um empate técnico nesse cenário, com a ressalva de que o FCFS entrega maior estabilidade e previsibilidade (menor variância) devido à ausência das interrupções de prioridade.

7.2. Resultados para testes de processos *IO-bound*

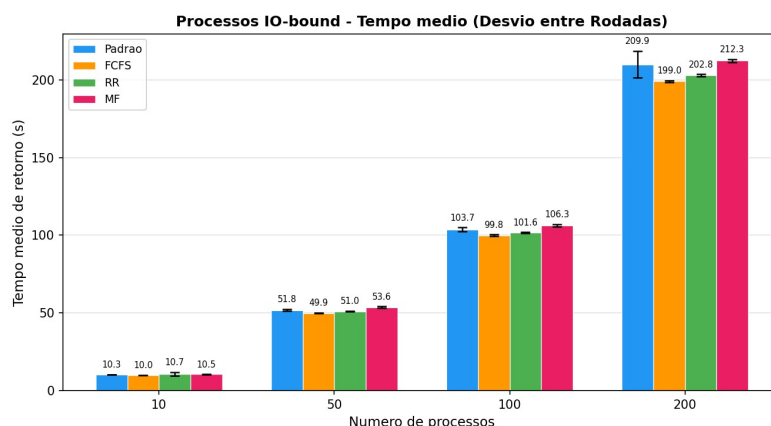


Figura 8. Resultados obtidos para processos *IO-bound*

A análise dos processos *IO-bound*, apresentada na Figura 8, revela tempos médios de retorno na ordem dos segundos. Isso reflete a natureza deste perfil de carga, que passa a maior parte de sua vida útil bloqueado, aguardando a conclusão de operações de entrada e saída.

De imediato, observa-se que o comportamento e o desvio-padrão dos algoritmos foram muito mais uniformes se comparados aos resultados *CPU-bound*. Apesar da magnitude em segundos, as barras de erro são extremamente curtas, indicando que o comportamento dos escalonadores para E/S é altamente previsível e consistente entre as rodadas.

Avaliando os algoritmos, é notável que o FCFS apresentou a melhor performance em todas as faixas de teste, com seu tempo médio variando de 10.0s (10 processos) até 199.0s (200 processos). Embora seja considerado um algoritmo teoricamente ineficiente, o comportamento observado é explicado pela eliminação da sobrecarga (*overhead*) do sistema operacional. Neste cenário, o Efeito Comboio não ocorre, pois todos os processos realizam *I/O* e bloqueiam voluntariamente de forma rápida. Ao remover os cálculos de envelhecimento e o trânsito dinâmico entre as 16 filas, o sistema apenas retira o processo da fila única e o devolve ao final dela após o *I/O*, poupando ciclos do microkernel.

Seguido do FCFS, o Round-Robin (RR) apresentou o segundo melhor desempenho geral para *I/O* intenso (202.8s no pior cenário). O tamanho do *quantum* tem pouco impacto neste caso, pois os processos bloqueiam por E/S antes de esgotarem suas fatias de tempo. Quando o processo fica pronto novamente, ele é enviado ao final da fila, tornando seu fluxo muito semelhante ao do FCFS, diferindo apenas pelo leve *overhead* de monitoramento do *timer* que o RR ainda mantém.

Por outro lado, algoritmos baseados em prioridades dinâmicas (Padrão do MINIX e MF) mostraram-se menos favoráveis para este perfil de carga. O algoritmo MF registrou o pior tempo (212.3s) com 200 processos. Projetado para beneficiar processos que usam muita CPU através do aumento do *quantum*, o MF não oferece vantagem para processos que bloqueiam constantemente. Na prática, a manipulação das múltiplas filas e as checagens constantes de prioridade provocam atrasos. O mesmo princípio se aplica ao algoritmo padrão do MINIX (209.9s), onde processos limitados por *I/O* são rebaixados com menos frequência e acabam congestionando as filas de alta prioridade, gerando um excesso de cálculos para o servidor de escalonamento tentar balancear uma carga que, fundamentalmente, só precisa aguardar a resposta dos periféricos.

Conclui-se que a aproximação dos tempos de retorno observada entre as diferentes abordagens ocorre porque, em cenários limitados por E/S, o peso de processamento do escalonador perde relevância. Nesses testes, o gargalo dominante do sistema passa a ser a espera pela resposta do hardware periférico, o que nivela os tempos absolutos de retorno, diminuindo o impacto do algoritmo de escalonamento adotado.

8. Conclusão

Este projeto teve como objetivo principal analisar os diferentes comportamentos de algoritmos clássicos de escalonamento e compará-los com o algoritmo padrão do MINIX. Através da coleta e análise dos tempos médios de execução e do desvio-padrão para processos CPU-bound e IO-bound, em diferentes quantidades de processos, foi possível completar nosso objetivo de analisar como as características de cada método afetam no desempenho da CPU.

O algoritmo padrão do MINIX 3, por exemplo, mesmo possuindo tempos médios de execução maiores que de outros métodos em certas situações, em algumas execuções ele foi capaz de atingir tempos médios muito menores que seus pares, demonstrando a capacidade do algoritmo padrão do SO. Para o FCFS, foi possível obter resultados satisfatórios, porém, em alguns experimentos, foi possível verificar um alto desvio-padrão, indicando que o algoritmo é bem instável quando comparado ao resto. O Round-Robin com *quantum* variável foi o pior desempenho deste experimento, principalmente em casos com muitos processos, já que, por conta da grande divisão do *quantum*, há uma quantidade excessiva de trocas de contexto. Por fim, o múltiplas filas com *quantum* exponencial teve resultados mistos, sendo capaz de desempenhar bem em processos CPU-bound, mas em casos de processos IO-bound, possuiu as piores performances.

Concluimos, portanto, que o algoritmo padrão do MINIX ainda se mostra como uma alternativa adequada para um sistema operacional que deve ser escalável, já que ele foi capaz de performar de maneira satisfatória em grande parte dos casos em que testamos, diferente dos outros métodos, sendo capaz, até, de superá-los em algumas execuções.

9. Divisão das tarefas

Tabela 1. Divisão de tarefas entre os membros do grupo

Integrante	Atividades
Breno C. R. Nakamura	Etapa #2: FCFS e primeiros testes; Etapa #3: Seções: 1, 6.1, 6.4, 7; Revisão Geral do Documento.
Estevan T. Santos	Execução dos Testes; Plotagem dos Gráficos; Elaboração do Vídeo para Entrega.
Gabriel L. Bonavina	Etapa #1; Etapa #2: Round-Robin e primeiros testes; Etapa #3: Seções: Resumo, 2, 3, 4, 5, 6.2 e 7.
Jonas L. Durão	Etapa #1; Etapa #2: Round-Robin e primeiros testes; Etapa #3: Seções: 2, 3, 4, 6.2 e 7.
Natália V. A. do Val	Etapa #2: FCFS e primeiros testes; Etapa #3: Seções: 1, 6.1, 6.4, 7; Revisão Geral do Documento.
Vinícius A. L. Andrade	Etapa #2: Múltiplas Filas com Quantum Exponencial e primeiros testes; Etapa #3: Seções: 6.3, 7 e 8; Revisão Geral do Documento.

Referências

- MINIX 3 Project. Minix 3 wiki documentation. <https://wiki.minix3.org/>. Accessed: 2026-06-04.
- Tanenbaum, A. S. (2016). *Sistemas Operacionais Modernos*. São Paulo: Pearson Education Brasil, 4th edition.
- Tanenbaum, A. S. and Woodhull, A. S. (2008). *Sistemas Operacionais: Projeto e Implementação*. Porto Alegre: Bookman, 3rd edition.